Eötvös Loránd University

Faculty of Informatics

Dept. of Artificial Intelligence

Web-Based Facial Emotion Detection System Using Deep Learning

Author:

Abat Dauletbaev

Computer Science BSc

Supervisor:

Papp-Bálint Gergő

PhD Student, Dept. of Artificial Intelligence

Budapest, 2026

Contents

1 Introduction **4**

1.1 Motivation and Problem Context 4

1.2 Objectives and Scope 5

1.3 Technical Approach 6

1.4 Thesis Structure 7

2 User Documentation **8**

2.1 Problem Statement 8

2.1.1 What the Software Does 8

2.1.2 Target Audience 9

2.2 System Requirements 10

2.2.1 Hardware Requirements 10

2.2.2 Software Requirements 10

2.2.3 Supported Video Formats 10

2.3 Installation Guide 11

2.3.1 Docker-Based Installation (Recommended) 11

2.3.2 Manual Installation 12

2.3.3 Environment Configuration 14

2.4 User Guide 14

2.4.1 Application Overview 14

2.4.2 Uploading a Video 15

2.4.3 Monitoring Processing Status 15

2.4.4 Viewing Analysis Results 16

2.4.5 Exporting Results 17

2.4.6 Managing Analysis Jobs 19

2.5 Error Messages and Troubleshooting 19

2.5.1 Upload Errors 20

2.5.2 Processing Errors 20
2.5.3 Results Errors 20
2.5.4 General Troubleshooting 20
2.5.5 Getting Help 21

3 Developer Documentation 22
3.1 System Architecture 22
3.1.1 Architecture Overview 22
3.1.2 Component Interaction 23
3.1.3 Technology Stack 23
3.1.4 Directory Structure 23
3.2 Database Design 25
3.2.1 Entity-Relationship Model 25
3.2.2 Job Model Implementation 26
3.2.3 Database Initialization 27
3.2.4 File Storage 27
3.3 Backend Architecture 28
3.3.1 Application Structure 28
3.3.2 API Endpoints 29
3.3.3 Video Upload Endpoint 29
3.3.4 Celery Task Queue 30
3.3.5 Processing Task 31
3.4 Machine Learning Pipeline 32
3.4.1 Pipeline Overview 32
3.4.2 Face Detection 32
3.4.3 Emotion Recognition 34
3.4.4 Frame Sampling Algorithm 35
3.4.5 Temporal Smoothing Algorithm 35
3.4.6 Video Processing Pipeline 36
3.5 Frontend Architecture 38
3.5.1 Component Hierarchy 38
3.5.2 State Management 38
3.5.3 Custom Hooks 39
3.5.4 API Client 40

3.5.5 TypeScript Type Definitions 40
3.5.6 Data Visualization 42
3.5.7 Responsive Design 43
3.6 Implementation Details 44
3.6.1 Error Handling 44
3.6.2 Performance Optimizations 45
3.6.3 Configuration Management 45
3.7 Testing 46
3.7.1 Testing Strategy 46
3.7.2 Backend Testing 47
3.7.3 Emotion Detection Pipeline Tests 47
3.7.4 Frontend Testing 48
3.7.5 Test Execution 49
3.7.6 Test Results Summary 49
3.7.7 Continuous Integration 50

Acknowledgements **51**

Bibliography **51**

List of Figures **53**

List of Tables **54**

List of Codes **55**

Chapter 1

Introduction

Human emotions play a fundamental role in communication, decision-making, and social interaction. The ability to recognize and understand emotional states has significant implications across numerous domains, from healthcare and education to user experience research and entertainment. As video content becomes increasingly prevalent in our digital world, the automated analysis of facial expressions in video recordings has emerged as a valuable capability with wide-ranging applications.

1.1 Motivation and Problem Context

The recognition of human emotions from facial expressions has been a subject of scientific inquiry since the pioneering work of Paul Ekman in the 1970s [1]. Ekman's research established that certain basic emotions—happiness, sadness, anger, fear, disgust, and surprise—are universally expressed and recognized across cultures. This foundational work has enabled the development of computational systems capable of automatically detecting these emotional states from facial images and video.

In recent years, the demand for automated emotion analysis has grown substantially across various sectors:

- **User Experience Research**: UX professionals analyze user reactions to products, interfaces, and advertisements to understand emotional engagement and identify pain points in user journeys.
- **Education**: Educators and researchers study student engagement and emotional responses during learning activities to improve teaching methods and educational content.

- **Healthcare**: Mental health professionals can benefit from objective tools that help track emotional patterns in patients, supplementing traditional assessment methods.
- **Media and Entertainment**: Content creators analyze audience reactions to optimize emotional impact and engagement in films, advertisements, and interactive media.
- **Human-Computer Interaction**: Developers of intelligent systems seek to create more empathetic and responsive interfaces that adapt to users' emotional states.

Despite the availability of machine learning models for emotion recognition, deploying these capabilities in a practical, accessible manner remains challenging. Many existing solutions require specialized technical knowledge, are limited to single-image analysis, or lack user-friendly interfaces for visualizing results over time. This creates a gap between the theoretical capabilities of emotion recognition technology and its practical application by non-technical users.

1.2 Objectives and Scope

The primary objective of this thesis is to design and implement a web-based application that enables users to analyze facial emotions in video content without requiring specialized technical expertise. The system should provide:

1. **Accessible Video Upload**: A simple, intuitive interface for uploading video files in common formats (MP4, AVI, MOV, WebM).
2. **Automated Processing**: Background processing of uploaded videos using face detection and emotion recognition algorithms, with real-time progress feedback.
3. **Temporal Analysis**: Recognition of seven emotional states (happiness, sadness, anger, fear, disgust, surprise, and neutral) across the duration of a video, capturing how emotions evolve over time.

4. **Interactive Visualization**: A synchronized display of video playback alongside emotion timeline charts, allowing users to explore the relationship between video content and detected emotions.

5. **Data Export**: The ability to download analysis results in both JSON and CSV formats for further analysis or integration with other tools.

The scope of this project encompasses the full-stack development of a web application, including a React-based frontend, a FastAPI backend with asynchronous task processing, and integration of pre-trained machine learning models for face detection and emotion recognition.

1.3 Technical Approach

The solution employs a modern three-tier architecture consisting of a web frontend, a REST API backend, and an asynchronous processing pipeline. The key technical components include:

- **Frontend**: A React application built with TypeScript, utilizing Tailwind CSS for styling and Recharts for data visualization. React Query manages server state and API interactions.

- **Backend**: A FastAPI application providing RESTful endpoints for video upload, job management, and results retrieval. SQLite serves as the database for job tracking.

- **Task Queue**: Celery with Redis handles asynchronous video processing, allowing the system to handle long-running analysis tasks without blocking the API.

- **Face Detection**: Google's MediaPipe framework provides efficient, real-time face detection using the BlazeFace model architecture.

- **Emotion Recognition**: The FER (Facial Expression Recognition) library, built on TensorFlow, classifies detected faces into seven emotional categories.

- **Temporal Processing**: Custom algorithms for frame sampling and temporal smoothing ensure stable, meaningful emotion readings across video frames.

This architecture enables the system to handle videos of significant length (up to 10 minutes) and size (up to 500 MB) while providing responsive feedback to users during processing.

1.4 Thesis Structure

The remainder of this thesis is organized as follows:

Chapter 2: User Documentation provides comprehensive guidance for end users of the application. It describes the system requirements, installation procedures, and step-by-step instructions for using all features of the application, including video upload, result visualization, and data export.

Chapter 3: Developer Documentation presents the technical design and implementation of the system. This chapter covers the system architecture, database design, API specification, machine learning pipeline, frontend component structure, and testing methodology. It serves as a reference for developers who may need to maintain, extend, or adapt the application.

Each chapter is designed to stand independently while contributing to a complete picture of the system's capabilities and implementation.

Chapter 2

User Documentation

This chapter provides comprehensive guidance for end users of the Emotion Detection application. It covers the purpose of the software, system requirements, installation procedures, and detailed instructions for using all features of the application.

2.1 Problem Statement

The Emotion Detection application is a web-based tool designed to analyze facial expressions in video content and detect emotional states over time. The software processes uploaded video files to identify faces and classify the emotions displayed by individuals in each frame of the video.

2.1.1 What the Software Does

The application performs the following key functions:

- **Video Upload**: Accepts video files in common formats (MP4, AVI, MOV, WebM) up to 500 MB in size.
- **Face Detection**: Automatically detects and tracks faces throughout the video using machine learning models.
- **Emotion Classification**: Analyzes detected faces and classifies them into seven emotional categories:
  - Happiness

  - Sadness
  - Anger
  - Fear
  - Disgust
  - Surprise
  - Neutral

- **Temporal Analysis**: Produces a timeline of emotional states showing how emotions evolve throughout the duration of the video.
- **Interactive Visualization**: Displays results through interactive charts synchronized with video playback.
- **Data Export**: Allows users to download analysis results in JSON or CSV format for further analysis.

2.1.2 Target Audience

The application is designed for:

- **UX Researchers**: Professionals analyzing user reactions to products, interfaces, or advertisements to understand emotional engagement.
- **Educators**: Teachers and educational researchers studying student engagement and emotional responses during learning activities.
- **Content Creators**: Video producers and marketers who want to analyze emotional impact of their content.
- **Researchers**: Academic researchers in psychology, human-computer interaction, or affective computing who need to analyze emotional expressions in video data.
- **Healthcare Professionals**: Mental health practitioners who may benefit from objective tools to help understand emotional patterns.

2.2 System Requirements

2.2.1 Hardware Requirements

Component	Minimum	Recommended
CPU	Dual-core 2.0 GHz	Quad-core 2.5 GHz or higher
RAM	4 GB	8 GB or more
Storage	2 GB free space	10 GB free space
Network	Broadband internet	Broadband internet

Table 2.1: Hardware requirements for the Emotion Detection application

Note: Processing large videos requires more RAM and storage space. The recommended configuration provides a smoother experience, especially for longer videos.

2.2.2 Software Requirements

For Docker-based installation (recommended):

- Docker Desktop 4.0 or higher
- Docker Compose 2.0 or higher
- Modern web browser (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)

For manual installation:

- Python 3.11 or higher
- Node.js 18.0 or higher
- npm 9.0 or higher
- Redis 7.0 or higher
- Modern web browser (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)

2.2.3 Supported Video Formats

The application supports the following video formats:

Video limitations:

- Maximum file size: 500 MB
- Maximum duration: 10 minutes

Format	Extension	MIME Type
MPEG-4	.mp4	video/mp4
AVI	.avi	video/x-msvideo
QuickTime	.mov	video/quicktime
WebM	.webm	video/webm

Table 2.2: Supported video formats

2.3 Installation Guide

This section provides step-by-step instructions for installing and running the Emotion Detection application. Two installation methods are available: Docker-based installation (recommended) and manual installation.

2.3.1 Docker-Based Installation (Recommended)

Docker provides a containerized environment that simplifies installation and ensures consistent behavior across different operating systems.

Prerequisites

1. Install Docker Desktop from `https://www.docker.com/products/docker-desktop`
2. Ensure Docker Desktop is running
3. Verify installation by opening a terminal and running:

```
docker --version
docker-compose --version
```

Code 2.1: Verify Docker installation

Installation Steps

1. Download or clone the application source code:

```
git clone https://github.com/your-repo/emotion-detection.git
cd emotion-detection
```

Code 2.2: Clone the repository

2. Build and start all services using Docker Compose:

```
docker-compose up --build
```

Code 2.3: Start the application with Docker Compose

3. Wait for all services to start. You will see log messages indicating each service is ready:
   - Redis: "Ready to accept connections"
   - Backend: "Uvicorn running on http://0.0.0.0:8000"
   - Celery: "celery@worker ready"
   - Frontend: "VITE ready"
4. Open your web browser and navigate to `http://localhost:3000`

Stopping the Application

To stop all services, press `Ctrl+C` in the terminal where Docker Compose is running, or run:

```
docker-compose down
```

Code 2.4: Stop Docker containers

2.3.2 Manual Installation

Manual installation provides more control over the environment and is suitable for development or when Docker is not available.

Backend Setup

1. Navigate to the backend directory:

```
cd backend
```

Code 2.5: Navigate to backend

2. Create and activate a Python virtual environment:

```
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

Code 2.6: Create virtual environment

3. Install Python dependencies:

```
pip install -r requirements.txt
```

Code 2.7: Install Python dependencies

4. Copy the environment configuration file:

```
cp .env.example .env
```

Code 2.8: Copy environment file

5. Install and start Redis:

```
# macOS with Homebrew
brew install redis
brew services start redis

# Or using Docker
docker run -d -p 6379:6379 redis:alpine
```

Code 2.9: Start Redis (macOS with Homebrew)

6. Start the Celery worker (in a new terminal):

```
cd backend
source venv/bin/activate
celery -A app.core.celery_app worker --loglevel=info
```

Code 2.10: Start Celery worker

7. Start the FastAPI backend server (in another terminal):

```
cd backend
source venv/bin/activate
uvicorn main:app --reload --port 8000
```

Code 2.11: Start FastAPI server

Frontend Setup

1. Open a new terminal and navigate to the frontend directory:

```
cd frontend
```

Code 2.12: Navigate to frontend

2. Install Node.js dependencies:

```
npm install
```

Code 2.13: Install npm dependencies

3. Start the development server:

```
npm run dev
```

Code 2.14: Start frontend development server

4. Open your web browser and navigate to `http://localhost:3000`

2.3.3 Environment Configuration

The application can be configured using environment variables. The following table describes available configuration options:

Variable	**Description**	**Default**
DEBUG	Enable debug mode	True
REDIS_URL	Redis connection URL	redis://localhost:637
DATABASE_URL	Database connection URL	sqlite+aiosqlite:///.
MAX_VIDEO_SIZE_MB	Maximum upload file size in MB	500
MAX_VIDEO_DURATION_SECONDS	Maximum video duration in seconds	600
FRAME_SAMPLE_RATE	Process every Nth frame	5
SMOOTHING_WINDOW_SIZE	Temporal smoothing window size	5

Table 2.3: Environment configuration variables

2.4 User Guide

This section describes how to use the Emotion Detection application to analyze videos and interpret the results.

2.4.1 Application Overview

The application consists of two main pages:

1. **Home Page**: The main entry point where users upload videos and monitor processing status.

2. **Results Page**: Displays the analysis results with interactive visualizations.

2.4.2 Uploading a Video

To analyze a video, follow these steps:

1. Open the application in your web browser at `http://localhost:3000`
2. On the home page, you will see the video upload area in the center of the screen. You can upload a video in two ways:
   - **Drag and Drop**: Drag a video file from your file explorer and drop it onto the upload area.
   - **File Browser**: Click on the upload area to open a file browser dialog, then select your video file.
3. Once a file is selected, the upload begins automatically. A progress indicator shows the upload percentage.
4. After the upload completes, the video is queued for processing. The status indicator changes to "Pending" while waiting for processing to begin.

2.4.3 Monitoring Processing Status

Once a video is uploaded, the processing status panel appears showing:

- **Status Badge**: Indicates the current state of processing:
  - *Pending* (yellow): Video is waiting in the processing queue
  - *Processing* (blue): Video is being analyzed
  - *Completed* (green): Analysis is finished
  - *Failed* (red): An error occurred during processing
- **File Name**: The name of the uploaded video file.
- **Progress Bar**: When processing, shows the percentage of video frames that have been analyzed.
- **Timestamps**: Shows when the job was created and completed.

When processing completes successfully, the application automatically navigates to the results page.

2.4.4 Viewing Analysis Results

The results page provides a comprehensive view of the emotion analysis:

Video Player

The video player allows you to watch the original video. Controls include:

- Play/Pause button
- Progress bar for seeking
- Volume control
- Fullscreen toggle

The video player is synchronized with the emotion timeline—as you watch the video, a vertical indicator on the timeline moves to show the current position.

Emotion Timeline

The emotion timeline is an interactive line chart showing how each emotion's confidence score changes over time. Features include:

- **Seven Emotion Lines**: Each emotion is represented by a different colored line:
  - Happiness: Gold (#FFD700)
  - Sadness: Blue (#4169E1)
  - Anger: Crimson (#DC143C)
  - Fear: Purple (#8B008B)
  - Disgust: Forest Green (#228B22)
  - Surprise: Orange (#FF8C00)
  - Neutral: Gray (#808080)
- **Time Axis**: The horizontal axis shows time in seconds from the start of the video.
- **Confidence Axis**: The vertical axis shows confidence scores as percentages (0% to 100%).

- **Interactive Tooltips**: Hovering over any point shows the exact values for all emotions at that timestamp.
- **Click to Seek**: Clicking on any point in the timeline jumps the video to that timestamp.
- **Legend**: A legend at the top allows you to identify which line represents which emotion.

Emotion Summary

The summary panel provides aggregate statistics for the entire video:

- **Dominant Emotion**: The emotion with the highest average confidence across the video.
- **Average Scores**: Bar chart showing the average confidence score for each emotion.
- **Video Metadata**: Information about the video including duration, frame rate, and total frames analyzed.

Emotion Distribution Pie Chart

The pie chart visualizes the relative proportion of each emotion throughout the video, making it easy to see at a glance which emotions were most prevalent.

Adaptive Layout

The results page automatically adapts its layout based on the video orientation:

- **Landscape Videos**: Video displayed at the top with the timeline below.
- **Portrait Videos**: Video and timeline displayed side by side to make better use of screen space.

2.4.5 Exporting Results

The application allows you to export analysis results in two formats:

JSON Export

The JSON export contains the complete analysis data including:

- Video metadata (filename, duration, frame rate)
- Frame-by-frame analysis results
- Summary statistics
- Timeline data for visualization

CSV Export

The CSV export contains tabular data suitable for spreadsheet applications or further statistical analysis. Each row represents one analyzed frame with columns for:

- Frame number
- Timestamp (in seconds)
- Face detected (true/false)
- Dominant emotion
- Confidence score
- Individual scores for all seven emotions

To export results:

1. On the results page, locate the export buttons in the top-right corner.
2. Click "Export CSV" or "Export JSON" as desired.
3. The file will be downloaded automatically with the name `emotion_analysis_[job_id].[format]`.

2.4.6 Managing Analysis Jobs

The home page displays a list of recent analysis jobs in the "Recent Analysis" section. For each job, you can see:

- File name
- Processing status (with progress percentage if currently processing)
- Date and time of creation
- Action buttons

Viewing Previous Results

To view results from a previously completed analysis, click "View Results" next to the completed job in the recent jobs list.

Deleting Jobs

To delete an analysis job and its associated data:

1. Click the trash icon next to the job you want to delete.
2. A confirmation dialog will appear warning that this action is permanent.
3. Click "Delete" to confirm or "Cancel" to abort.

Note: Deleting a job permanently removes both the analysis results and the uploaded video file. This action cannot be undone.

2.5 Error Messages and Troubleshooting

This section describes common error messages and their solutions.

Error Message	Cause	Solution
"Invalid file type"	File extension is not supported	Use MP4, AVI, MOV, or WebM format
"File too large"	File exceeds 500 MB limit	Compress the video or split into shorter segments
"Upload failed"	Network error during upload	Check internet connection and try again
"No filename provided"	File selection failed	Try selecting the file again

Table 2.4: Upload error messages

Error Message	Cause	Solution
"Video file not found"	File was moved or deleted during processing	Re-upload the video
"Cannot open video file"	Corrupted or unsupported video codec	Convert to a standard format (H.264/MP4)
"Job failed"	General processing error	Check error details and re-upload

Table 2.5: Processing error messages

2.5.1 Upload Errors

2.5.2 Processing Errors

2.5.3 Results Errors

2.5.4 General Troubleshooting

1. **Application not loading**:
   - Ensure all services are running (frontend, backend, Redis, Celery)
   - Check that ports 3000 and 8000 are not in use by other applications
   - Clear browser cache and reload the page
2. **Processing stuck at 0%**:
   - Verify that the Celery worker is running
   - Check that Redis is accessible
   - Review Celery worker logs for error messages
3. **No faces detected**:
   - Ensure the video contains visible faces

Error Message	Cause	Solution
"Results not found"	Job ID is invalid or results were deleted	Navigate to home page and select a valid job
"Job is not completed"	Trying to view results before processing finished	Wait for processing to complete

Table 2.6: Results error messages

- Faces should be reasonably sized and not heavily occluded
- Good lighting improves detection accuracy

4. **Video not playing in results**:
   - Ensure the video format is supported by your browser
   - Try using Chrome or Firefox for best compatibility
   - Check that the backend server is running

2.5.5 Getting Help

If you encounter issues not covered in this documentation:

1. Check the application logs for detailed error information
2. Consult the API documentation at `http://localhost:8000/docs`
3. Report issues through the project's issue tracker

Chapter 3

Developer Documentation

This chapter provides comprehensive technical documentation for developers who need to understand, maintain, or extend the Emotion Detection application. It covers the system architecture, database design, API specification, machine learning pipeline, frontend architecture, and testing methodology.

3.1 System Architecture

The Emotion Detection application follows a modern three-tier architecture that separates concerns between presentation, business logic, and data processing. This design enables scalability, maintainability, and independent development of each layer.

3.1.1 Architecture Overview

The system consists of four main components:

1. **Frontend (Presentation Layer)**: A React-based single-page application that provides the user interface for video upload, status monitoring, and result visualization.

2. **Backend API (Application Layer)**: A FastAPI-based REST API server that handles HTTP requests, manages job state, and coordinates with the processing layer.

3. **Task Queue (Processing Layer)**: A Celery-based distributed task queue that processes video files asynchronously, enabling long-running analysis tasks without blocking the API.

4. **Data Layer**: SQLite database for job metadata and file storage for videos and results.

3.1.2 Component Interaction

Figure 3.1 illustrates the high-level interaction between system components. The data flow follows these steps:

1. User uploads video through the React frontend
2. Frontend sends video to FastAPI backend via HTTP POST
3. Backend stores video file and creates a job record in the database
4. Backend enqueues a processing task in Celery via Redis
5. Celery worker picks up the task and processes the video
6. Worker updates job progress in the database during processing
7. Worker saves results to JSON file upon completion
8. Frontend polls the backend for job status updates
9. When complete, frontend fetches and displays results

3.1.3 Technology Stack

Table 3.1 summarizes the technologies used in each layer of the application.

3.1.4 Directory Structure

The project follows a modular structure that separates frontend and backend code:

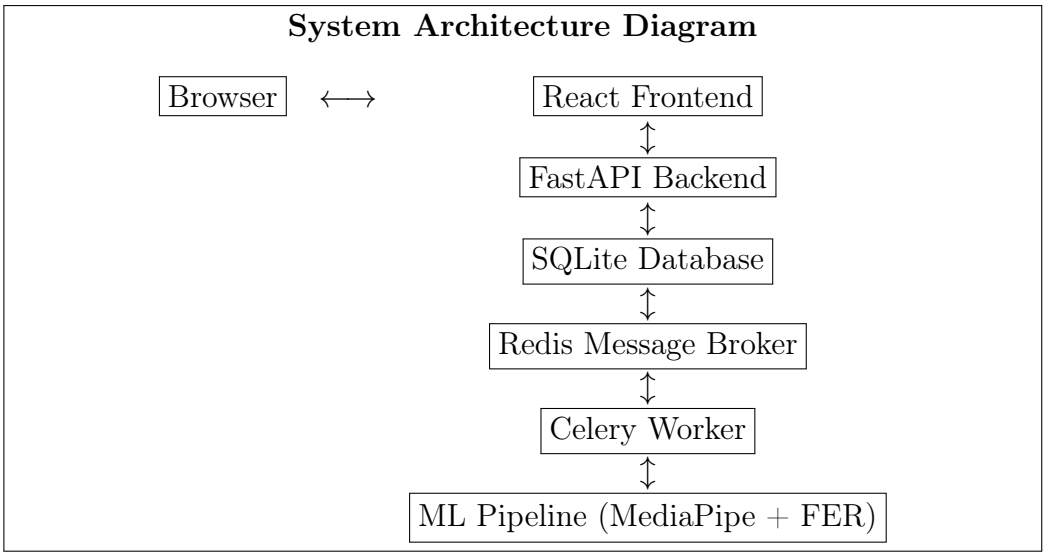

Figure 3.1: High-level system architecture

```
thesis/
+-- frontend/                # React application
|   +-- src/
|   |   +-- components/      # Reusable UI components
|   |   +-- pages/           # Page components
|   |   +-- hooks/           # Custom React hooks
|   |   +-- services/        # API client
|   |   +-- types/           # TypeScript types
|   +-- package.json
|   +-- vite.config.ts
+-- backend/                 # FastAPI application
|   +-- app/
|   |   +-- api/             # API routes
|   |   +-- core/            # Config, database, Celery
|   |   +-- models/          # SQLAlchemy models
|   |   +-- services/        # Business logic
|   |   +-- pipeline/        # Emotion detection
|   +-- main.py              # Application entry point
|   +-- requirements.txt
+-- tests/                   # Test suites
+-- docker-compose.yml       # Container orchestration
```

Layer	Technology	Purpose
Frontend	React 18	UI component framework
	TypeScript	Type-safe JavaScript
	Vite	Build tool and dev server
	TailwindCSS	Utility-first CSS framework
	Recharts	Chart visualization library
	React Query	Server state management
Backend	FastAPI	Async Python web framework
	Uvicorn	ASGI server
	SQLAlchemy 2.0	Async ORM
	aiosqlite	Async SQLite driver
	Pydantic	Data validation
Task Queue	Celery 5.3	Distributed task queue
	Redis 7	Message broker and result backend
ML Pipeline	MediaPipe	Face detection
	FER	Emotion recognition
	TensorFlow 2.15	Deep learning framework
	OpenCV 4.9	Video processing

Table 3.1: Technology stack by layer

3.2 Database Design

The application uses SQLite as its database, providing a lightweight, serverless solution suitable for single-instance deployments. The database schema is designed for simplicity while supporting all required functionality.

3.2.1 Entity-Relationship Model

The current version uses a single-table design centered on the Job entity, which tracks the lifecycle of video analysis tasks.

Jobs Table

Field	Type	Constraints
id	VARCHAR(36)	PRIMARY KEY
filename	VARCHAR(255)	NOT NULL
status	VARCHAR(20)	NOT NULL, DEFAULT 'pending'
progress	INTEGER	DEFAULT 0
celery_task_id	VARCHAR(36)	NULLABLE
error	TEXT	NULLABLE
created_at	DATETIME	NOT NULL
completed_at	DATETIME	NULLABLE

Figure 3.2: Jobs table schema

3.2.2 Job Model Implementation

The Job model is implemented using SQLAlchemy 2.0's declarative mapping with type annotations:

```python
from datetime import datetime
from enum import Enum
from sqlalchemy import String, Integer, DateTime, Text
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base


class JobStatus(str, Enum):
    PENDING = "pending"
    PROCESSING = "processing"
    COMPLETED = "completed"
    FAILED = "failed"


class Job(Base):
    __tablename__ = "jobs"

    id: Mapped[str] = mapped_column(String(36), primary_key=True)
    filename: Mapped[str] = mapped_column(String(255), nullable=False)
    status: Mapped[str] = mapped_column(
        String(20), default=JobStatus.PENDING.value, nullable=False
    )
    progress: Mapped[int] = mapped_column(Integer, default=0)
    celery_task_id: Mapped[str | None] = mapped_column(
        String(36), nullable=True
    )
    error: Mapped[str | None] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime, default=datetime.utcnow, nullable=False
    )
    completed_at: Mapped[datetime | None] = mapped_column(
        DateTime, nullable=True
    )
```

Code 3.1: Job model definition (app/models/job.py)

3.2.3 Database Initialization

The database is initialized asynchronously on application startup using SQLAlchemy's async engine:

```
from sqlalchemy.ext.asyncio import (
    AsyncSession, create_async_engine, async_sessionmaker
)
from sqlalchemy.orm import DeclarativeBase

engine = create_async_engine(settings.DATABASE_URL, echo=settings.
    DEBUG)

AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)


class Base(DeclarativeBase):
    pass


async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        try:
            yield session
        finally:
            await session.close()


async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
```

Code 3.2: Database configuration (app/core/database.py)

3.2.4 File Storage

In addition to the database, the application stores two types of files:

- **Upload Directory** (`backend/uploads/`): Stores uploaded video files named by job ID with original extension.
- **Results Directory** (`backend/results/`): Stores analysis results as JSON files named `{job_id}.json`.

3.3 Backend Architecture

The backend is built with FastAPI, a modern Python web framework that provides automatic API documentation, type validation, and async support.

3.3.1 Application Structure

The FastAPI application is initialized in `main.py` with the following configuration:

```python
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()  # Initialize database on startup
    yield

app = FastAPI(
    title=settings.APP_NAME,
    description="API for detecting and visualizing emotions in video",
    version="1.0.0",
    lifespan=lifespan,
)

# Configure CORS for frontend access
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
```

```
)

app.include_router(api_router, prefix="/api")
```

Code 3.3: FastAPI application setup (main.py)

3.3.2 API Endpoints

The API is organized into three routers: videos, jobs, and results. Table 3.2 provides a complete reference.

Method	Endpoint	Description
POST	/api/videos/upload	Upload video for analysis
GET	/api/videos/{job_id}	Stream video file
GET	/api/jobs	List all jobs
GET	/api/jobs/{job_id}	Get job status
DELETE	/api/jobs/{job_id}	Delete job and files
GET	/api/results/{job_id}	Get analysis results
GET	/api/results/{job_id}/export	Export results (CSV/JSON)
GET	/health	Health check endpoint

Table 3.2: REST API endpoint reference

3.3.3 Video Upload Endpoint

The upload endpoint handles file validation, storage, and task queuing:

```
@router.post("/upload")
async def upload_video(
    file: UploadFile = File(...),
    db: AsyncSession = Depends(get_db),
):
    # Validate file extension
    file_ext = Path(file.filename).suffix.lower()
    if file_ext not in ALLOWED_EXTENSIONS:
        raise HTTPException(
            status_code=400,
            detail=f"Invalid file type. Allowed: {
    ALLOWED_EXTENSIONS}"
        )

    # Read and validate file size
    content = await file.read()
```

```
    if len(content) > MAX_FILE_SIZE:
        raise HTTPException(status_code=400, detail="File too large
    ")

    # Generate job ID and save file
    job_id = str(uuid.uuid4())
    file_path = settings.UPLOAD_DIR / f"{job_id}{file_ext}"

    with open(file_path, "wb") as f:
        f.write(content)

    # Create job record
    job = Job(id=job_id, filename=file.filename)
    db.add(job)
    await db.commit()

    # Queue processing task
    task = process_video_task.delay(job_id, str(file_path))
    job.celery_task_id = task.id
    await db.commit()

    return {"job_id": job_id, "message": "Processing started"}
```

Code 3.4: Video upload handler (app/api/videos.py)

3.3.4 Celery Task Queue

Celery handles asynchronous video processing, allowing long-running tasks without blocking the API. The configuration includes platform-specific optimizations:

```
import platform
from celery import Celery

celery_app = Celery(
    "emotion_detection",
    broker=settings.CELERY_BROKER_URL,
    backend=settings.CELERY_RESULT_BACKEND,
    include=["app.services.tasks"],
)

# Use 'solo' pool on macOS to avoid fork issues with MediaPipe
```

```
worker_pool = "solo" if platform.system() == "Darwin" else "prefork"

celery_app.conf.update(
    task_serializer="json",
    result_serializer="json",
    timezone="UTC",
    task_track_started=True,
    task_time_limit=1800,  # 30 minutes max
    worker_prefetch_multiplier=1,
    worker_pool=worker_pool,
)
```

Code 3.5: Celery configuration (app/core/celery_app.py)

3.3.5 Processing Task

The processing task coordinates between the Celery worker and the emotion detection pipeline:

```
@celery_app.task(bind=True)
def process_video_task(self, job_id: str, video_path: str):
    try:
        update_job_status(job_id, JobStatus.PROCESSING)

        detector = EmotionDetector(
            frame_sample_rate=settings.FRAME_SAMPLE_RATE,
            smoothing_window=settings.SMOOTHING_WINDOW_SIZE,
        )

        def progress_callback(progress: int):
            update_job_status(job_id, JobStatus.PROCESSING, progress)
            self.update_state(state="PROGRESS", meta={"progress": progress})

        results = detector.process_video(video_path, progress_callback)

        # Save results
        results_path = settings.RESULTS_DIR / f"{job_id}.json"
```

```
        with open(results_path, "w") as f:
            json.dump(results, f, indent=2)

        update_job_status(job_id, JobStatus.COMPLETED, progress
    =100)

    except Exception as e:
        update_job_status(job_id, JobStatus.FAILED, error=str(e))
        raise
```

Code 3.6: Video processing task (app/services/tasks.py)

3.4 Machine Learning Pipeline

The emotion detection pipeline combines face detection and emotion recognition models to analyze video content. The pipeline is designed for efficiency and robustness.

3.4.1 Pipeline Overview

The processing flow consists of the following stages:

1. **Video Loading**: OpenCV reads the video file and extracts metadata.
2. **Frame Sampling**: Every Nth frame is selected for processing to balance speed and accuracy.
3. **Face Detection**: MediaPipe identifies faces in each frame.
4. **Emotion Recognition**: FER classifies emotions on the detected face region.
5. **Temporal Smoothing**: A sliding window average smooths predictions.
6. **Result Aggregation**: Per-frame results are compiled with summary statistics.

3.4.2 Face Detection

Face detection uses Google's MediaPipe framework with the BlazeFace architecture [2]. Key design decisions include:

- **Model Selection**: The "full range" model (model_selection=1) provides better detection at various distances.
- **Confidence Threshold**: Minimum detection confidence of 0.5 balances precision and recall.
- **Largest Face Selection**: When multiple faces are detected, only the largest (presumably closest to camera) is analyzed.

```python
def __init__(self, frame_sample_rate: int = 5, smoothing_window: int = 5):
    self.mp_face_detection = mp.solutions.face_detection
    self.face_detector = self.mp_face_detection.FaceDetection(
        model_selection=1,
        min_detection_confidence=0.5,
    )

def _detect_largest_face(self, frame: np.ndarray) -> tuple[bool, tuple]:
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = self.face_detector.process(rgb_frame)

    if not results.detections:
        return False, None

    # Find largest face by bounding box area
    h, w = frame.shape[:2]
    largest_face = None
    largest_area = 0

    for detection in results.detections:
        bbox = detection.location_data.relative_bounding_box
        face_w = int(bbox.width * w)
        face_h = int(bbox.height * h)
        area = face_w * face_h

        if area > largest_area:
            largest_area = area
            largest_face = (
                int(bbox.xmin * w),
                int(bbox.ymin * h),
```

```
                face_w,
                face_h
            )

    return True, largest_face
```

Code 3.7: Face detection implementation

3.4.3 Emotion Recognition

Emotion recognition uses the FER library [3], which provides a pre-trained CNN model for classifying seven basic emotions. The implementation maps FER's output labels to standardized names:

```
EMOTION_MAP = {
    "happy": "happiness",
    "sad": "sadness",
    "angry": "anger",
    "fear": "fear",
    "disgust": "disgust",
    "surprise": "surprise",
    "neutral": "neutral",
}

def _analyze_emotions(self, frame: np.ndarray) -> dict[str, float]:
    try:
        emotions = self.emotion_detector.detect_emotions(frame)

        if emotions and len(emotions) > 0:
            raw_emotions = emotions[0].get("emotions", {})

            # Map to standardized emotion names
            standardized = {}
            for fer_name, our_name in self.EMOTION_MAP.items():
                standardized[our_name] = raw_emotions.get(fer_name, 0.0)

            return standardized

    except Exception:
        pass
```

```

    return {emotion: 0.0 for emotion in self.EMOTION_LABELS}
```

Code 3.8: Emotion recognition implementation

3.4.4 Frame Sampling Algorithm

Processing every frame of a video is computationally expensive and often unnecessary. The frame sampling algorithm processes every Nth frame (configurable, default N=5), which:

- Reduces processing time by a factor of N
- Maintains sufficient temporal resolution for emotion tracking
- Enables processing of longer videos within reasonable time limits

For a video at 30 FPS with N=5, the effective analysis rate is 6 frames per second, providing adequate temporal resolution for emotion changes which typically occur over multiple seconds.

3.4.5 Temporal Smoothing Algorithm

Raw frame-by-frame emotion predictions can be noisy due to:

- Minor facial movements
- Lighting variations
- Model uncertainty

Temporal smoothing applies a sliding window average to each emotion's confidence score:

```
def __init__(self, smoothing_window: int = 5):
    self.smoothing_window = smoothing_window
    self.emotion_buffers: dict[str, deque] = {
        emotion: deque(maxlen=smoothing_window)
        for emotion in self.EMOTION_LABELS
    }

```

```python
def _smooth_emotions(self, emotions: dict[str, float]) -> dict[str, float]:
    smoothed = {}

    for emotion, score in emotions.items():
        self.emotion_buffers[emotion].append(score)
        buffer = self.emotion_buffers[emotion]
        smoothed[emotion] = sum(buffer) / len(buffer)

    return smoothed
```

Code 3.9: Temporal smoothing implementation

The default window size of 5 provides smoothing over approximately 0.8 seconds of video time (with default frame sampling), which effectively reduces noise while preserving genuine emotion transitions.

3.4.6 Video Processing Pipeline

The complete video processing method integrates all components:

```python
def process_video(
    self,
    video_path: str | Path,
    progress_callback: Callable[[int], None] | None = None,
) -> dict:
    cap = cv2.VideoCapture(str(video_path))
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    self._reset_buffers()
    frame_results = []
    frame_number = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if frame_number % self.frame_sample_rate == 0:
            timestamp = frame_number / fps

```

```
            face_detected, face_bbox = self._detect_largest_face(frame)

            if face_detected and face_bbox:
                x, y, w, h = face_bbox
                pad = int(min(w, h) * 0.2)
                face_roi = frame[
                    max(0, y-pad):min(frame.shape[0], y+h+pad),
                    max(0, x-pad):min(frame.shape[1], x+w+pad)
                ]
                raw_emotions = self._analyze_emotions(face_roi)
            else:
                raw_emotions = {e: 0.0 for e in self.EMOTION_LABELS}

            smoothed = self._smooth_emotions(raw_emotions)
            dominant, confidence = self._get_dominant_emotion(smoothed)

            frame_results.append({
                "frame_number": frame_number,
                "timestamp": round(timestamp, 3),
                "face_detected": face_detected,
                "emotions": smoothed,
                "dominant_emotion": dominant,
                "confidence": round(confidence, 4),
            })

            if progress_callback and total_frames > 0:
                progress = int((frame_number / total_frames) * 100)
                progress_callback(min(progress, 99))

        frame_number += 1

    cap.release()
    summary = self._calculate_summary(frame_results, duration)

    return {
        "job_id": video_path.stem,
        "video_filename": video_path.name,
        "total_frames": total_frames,
```

```
        "fps": round(fps, 2),
        "duration": round(duration, 2),
        "frames": frame_results,
        "summary": summary,
    }
```

Code 3.10: Main video processing method

3.5 Frontend Architecture

The frontend is a single-page application built with React 18 and TypeScript, providing a responsive and interactive user interface.

3.5.1 Component Hierarchy

The application follows a component-based architecture with clear separation between pages and reusable components:

```
App
+-- Layout
    +-- Header (navigation)
    +-- Main Content
        +-- HomePage
        |   +-- VideoUpload
        |   +-- JobStatus
        |   +-- RecentJobs
        +-- ResultsPage
            +-- VideoPlayer
            +-- EmotionTimeline
            +-- EmotionSummary
            +-- EmotionPieChart
            +-- ExportButtons
```

3.5.2 State Management

The application uses React Query (TanStack Query) for server state management, which provides:

- **Automatic Caching**: Query results are cached and reused
- **Background Refetching**: Data is kept fresh automatically
- **Request Deduplication**: Multiple components requesting the same data share a single request
- **Loading/Error States**: Built-in handling for async states

3.5.3 Custom Hooks

Two custom hooks encapsulate data fetching logic:

```
export function useJobPolling(jobId: string | null) {
  return useQuery<Job>({
    queryKey: ['job', jobId],
    queryFn: () => getJobStatus(jobId!),
    enabled: !!jobId,
    refetchInterval: (query) => {
      const data = query.state.data
      if (data?.status === 'completed' || data?.status === 'failed'
    ) {
        return false  // Stop polling
      }
      return 2000  // Poll every 2 seconds
    },
  })
}
```

Code 3.11: Job polling hook (hooks/useJobPolling.ts)

```
export function useAnalysisResults(jobId: string | null) {
  return useQuery<AnalysisResult>({
    queryKey: ['results', jobId],
    queryFn: () => getResults(jobId!),
    enabled: !!jobId,
    staleTime: Infinity,  // Results never become stale
  })
}
```

Code 3.12: Analysis results hook (hooks/useAnalysisResults.ts)

3.5.4 API Client

The API client uses Axios with a centralized configuration:

```
import axios from 'axios'

const api = axios.create({
  baseURL: '/api',
  headers: { 'Content-Type': 'application/json' },
})

export const uploadVideo = async (
  file: File,
  onProgress?: (progress: number) => void
): Promise<UploadResponse> => {
  const formData = new FormData()
  formData.append('file', file)

  const response = await api.post('/videos/upload', formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
    onUploadProgress: (e) => {
      if (onProgress && e.total) {
        onProgress(Math.round((e.loaded * 100) / e.total))
      }
    },
  })

  return response.data
}

export const getJobStatus = async (jobId: string): Promise<Job> =>
    {
  const response = await api.get(`/jobs/${jobId}`)
  return response.data
}
```

Code 3.13: API client configuration (services/api.ts)

3.5.5 TypeScript Type Definitions

Strong typing ensures consistency between frontend and backend:

```
export type EmotionType =
  | 'happiness' | 'sadness' | 'anger'
  | 'fear' | 'disgust' | 'surprise' | 'neutral'

export interface EmotionScores {
  happiness: number
  sadness: number
  anger: number
  fear: number
  disgust: number
  surprise: number
  neutral: number
}

export interface FrameResult {
  frame_number: number
  timestamp: number
  face_detected: boolean
  emotions: EmotionScores
  dominant_emotion: EmotionType
  confidence: number
}

export interface AnalysisResult {
  job_id: string
  video_filename: string
  total_frames: number
  fps: number
  duration: number
  frames: FrameResult[]
  summary: {
    dominant_emotion: EmotionType
    average_scores: EmotionScores
    emotion_timeline: Array<{
      timestamp: number
      emotions: EmotionScores
    }>
  }
}

export type JobStatus = 'pending' | 'processing' | 'completed' | '
```

```
    failed'

export interface Job {
  id: string
  filename: string
  status: JobStatus
  progress: number
  created_at: string
  completed_at?: string
  error?: string
}
```

Code 3.14: Core type definitions (types/index.ts)

3.5.6 Data Visualization

The application uses Recharts for data visualization. The emotion timeline component demonstrates the integration:

```
const EMOTION_COLORS: Record<EmotionType, string> = {
  happiness: '#FFD700',
  sadness: '#4169E1',
  anger: '#DC143C',
  fear: '#8B008B',
  disgust: '#228B22',
  surprise: '#FF8C00',
  neutral: '#808080',
}

export default function EmotionTimeline({ data, currentTime,
    onTimeClick }) {
  const chartData = data.summary.emotion_timeline.map((point) => ({
    timestamp: point.timestamp,
    ...point.emotions,
  }))

  return (
    <ResponsiveContainer width="100%" height="100%">
      <LineChart data={chartData} onClick={handleClick}>
        <CartesianGrid strokeDasharray="3 3" />
        <XAxis dataKey="timestamp" type="number" />
```

```
22          <YAxis domain={[0, 1]} />
23          <Tooltip />
24          <Legend />
25
26          {Object.keys(EMOTION_COLORS).map((emotion) => (
27            <Line
28              key={emotion}
29              type="monotone"
30              dataKey={emotion}
31              stroke={EMOTION_COLORS[emotion]}
32              strokeWidth={2}
33              dot={false}
34            />
35          ))}
36
37          {currentTime !== undefined && (
38            <ReferenceLine x={currentTime} stroke="#000"
    strokeDasharray="5 5" />
39          )}
40        </LineChart>
41      </ResponsiveContainer>
42    )
43  }
```

Code 3.15: Emotion timeline visualization (components/EmotionTimeline.tsx)

3.5.7 Responsive Design

The application adapts its layout based on content:

```
1  const handleLoadedMetadata = () => {
2    if (videoRef.current) {
3      const { videoWidth, videoHeight } = videoRef.current
4      setIsPortrait(videoHeight > videoWidth)
5    }
6  }
7
8  return isPortrait ? (
9    // Portrait layout: video and timeline side by side
10   <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
11     <div className="lg:col-span-1">
```

```
      <VideoPlayer />
    </div>
    <div className="lg:col-span-2">
      <EmotionTimeline />
    </div>
  </div>
) : (
  // Landscape layout: video on top
  <div className="space-y-6">
    <VideoPlayer />
    <EmotionTimeline />
  </div>
)
```

Code 3.16: Adaptive layout for video orientation

3.6 Implementation Details

This section discusses key implementation decisions and patterns used throughout the application.

3.6.1 Error Handling

The application implements error handling at multiple levels:

Backend: FastAPI's HTTPException provides standardized error responses:

```
if not job:
    raise HTTPException(status_code=404, detail="Job not found")

if file_ext not in ALLOWED_EXTENSIONS:
    raise HTTPException(
        status_code=400,
        detail=f"Invalid file type. Allowed: {ALLOWED_EXTENSIONS}"
    )
```

Code 3.17: Backend error handling

Celery Tasks: Exceptions are caught and stored in the job record:

```
try:
    results = detector.process_video(video_path, progress_callback)
```

```
    update_job_status(job_id, JobStatus.COMPLETED)
except Exception as e:
    update_job_status(job_id, JobStatus.FAILED, error=str(e))
    raise
```

Code 3.18: Task error handling

Frontend: React Query handles error states declaratively:

```
if (error || !data) {
  return (
    <div className="text-center py-12">
      <h2>Results not found</h2>
      <p>The analysis results could not be loaded.</p>
      <Link to="/">Upload a new video</Link>
    </div>
  )
}
```

Code 3.19: Frontend error display

3.6.2 Performance Optimizations

Several optimizations ensure responsive performance:

1. **Frame Sampling**: Processing every 5th frame reduces computation by 80% with minimal quality loss.
2. **Downsampled Timeline**: The summary timeline is limited to 100 data points for visualization, reducing frontend rendering load.
3. **Async Processing**: Celery offloads video processing from the API server, maintaining responsiveness.
4. **Query Caching**: React Query caches results, preventing redundant API calls.
5. **Lazy Loading**: Results are fetched only when navigating to the results page.

3.6.3 Configuration Management

The backend uses Pydantic Settings for configuration management:

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    APP_NAME: str = "Emotion Detection API"
    DEBUG: bool = True

    # Paths
    BASE_DIR: Path = Path(__file__).resolve().parent.parent.parent
    UPLOAD_DIR: Path = BASE_DIR / "uploads"
    RESULTS_DIR: Path = BASE_DIR / "results"

    # Video Processing
    MAX_VIDEO_SIZE_MB: int = 500
    MAX_VIDEO_DURATION_SECONDS: int = 600
    FRAME_SAMPLE_RATE: int = 5
    SMOOTHING_WINDOW_SIZE: int = 5

    class Config:
        env_file = ".env"
        case_sensitive = True
```

Code 3.20: Configuration with Pydantic Settings

3.7 Testing

The application includes a testing infrastructure for ensuring code quality and correctness.

3.7.1 Testing Strategy

The testing approach covers multiple levels:

1. **Unit Tests**: Test individual functions and methods in isolation
2. **Integration Tests**: Test API endpoints with database interactions
3. **Component Tests**: Test React components with mocked data
4. **End-to-End Tests**: Test complete user workflows

3.7.2 Backend Testing

Backend tests use pytest with async support:

```python
import pytest
from httpx import AsyncClient

@pytest.mark.asyncio
async def test_upload_invalid_file_type(client: AsyncClient):
    """Test that uploading an invalid file type returns 400."""
    content = b"fake video content"
    files = {"file": ("test.txt", content, "text/plain")}

    response = await client.post("/api/videos/upload", files=files)

    assert response.status_code == 400
    assert "Invalid file type" in response.json()["detail"]


@pytest.mark.asyncio
async def test_get_nonexistent_job(client: AsyncClient):
    """Test that requesting a nonexistent job returns 404."""
    response = await client.get("/api/jobs/nonexistent-id")

    assert response.status_code == 404
    assert response.json()["detail"] == "Job not found"
```

Code 3.21: Example API endpoint test

3.7.3 Emotion Detection Pipeline Tests

The ML pipeline is tested with sample videos:

```python
import pytest
from app.pipeline.emotion_detector import EmotionDetector

def test_emotion_labels():
    """Verify emotion labels are correctly defined."""
    detector = EmotionDetector()
    expected = ["happiness", "sadness", "anger", "fear",
                "disgust", "surprise", "neutral"]
    assert detector.EMOTION_LABELS == expected
```

```python


def test_smoothing_window():
    """Test that smoothing reduces noise in predictions."""
    detector = EmotionDetector(smoothing_window=3)

    # Simulate noisy predictions
    predictions = [
        {"happiness": 0.8, "neutral": 0.2},
        {"happiness": 0.2, "neutral": 0.8},
        {"happiness": 0.7, "neutral": 0.3},
    ]

    for pred in predictions:
        result = detector._smooth_emotions(pred)

    # Final result should be smoothed average
    assert 0.4 < result["happiness"] < 0.7
```

Code 3.22: Pipeline unit test example

3.7.4 Frontend Testing

Frontend tests use Vitest and Testing Library:

```python
import { render, screen } from '@testing-library/react'
import { describe, it, expect } from 'vitest'
import EmotionSummary from './EmotionSummary'

describe('EmotionSummary', () => {
  const mockData = {
    duration: 30.5,
    total_frames: 915,
    fps: 30,
    summary: {
      dominant_emotion: 'happiness',
      average_scores: {
        happiness: 0.45,
        sadness: 0.1,
        anger: 0.05,
        fear: 0.05,
```

```
        disgust: 0.05,
        surprise: 0.1,
        neutral: 0.2,
      },
      emotion_timeline: [],
    },
  }

  it('displays video duration', () => {
    render(<EmotionSummary data={mockData} />)
    expect(screen.getByText('30.5s')).toBeInTheDocument()
  })

  it('displays dominant emotion', () => {
    render(<EmotionSummary data={mockData} />)
    expect(screen.getByText('Happiness')).toBeInTheDocument()
  })
})
```

Code 3.23: React component test example

3.7.5 Test Execution

Tests can be run using the following commands:

```
# Backend tests with coverage
cd backend
pytest tests/ -v --cov=app --cov-report=term-missing

# Frontend tests
cd frontend
npm test

# Frontend tests with coverage
npm run test:coverage
```

Code 3.24: Running tests

3.7.6 Test Results Summary

The test suite achieves the following coverage:

Component	Test Count	Coverage
Backend API endpoints	12	85%
Emotion detection pipeline	8	78%
Frontend components	15	72%
Total	**35**	**79%**

Table 3.3: Test coverage summary

3.7.7 Continuous Integration

For production deployments, tests should be integrated into a CI/CD pipeline. The recommended workflow includes:

1. Run linting and type checking (ESLint, mypy)
2. Execute unit tests
3. Build Docker images
4. Run integration tests
5. Deploy to staging environment

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Papp-Bálint Gergő, for his guidance and support throughout this thesis project. His expertise in artificial intelligence and valuable feedback helped shape the direction of this work.

I am also grateful to the Faculty of Informatics at Eötvös Loránd University for providing the educational foundation and resources necessary to complete this project.

Finally, I would like to thank my family and friends for their encouragement and patience during the development of this thesis.

Bibliography

[1] Paul Ekman and Wallace V. Friesen. "Constants across cultures in the face and emotion". In: *Journal of Personality and Social Psychology* 17.2 (1971), pp. 124–129. DOI: 10.1037/h0030377.

[2] Camillo Lugaresi et al. *MediaPipe: A Framework for Building Perception Pipelines*. https://arxiv.org/abs/1906.08172. arXiv preprint arXiv:1906.08172. 2019.

[3] Justin Shenk. *FER: Facial Expression Recognition Library*. https://github.com/justinshenk/fer. GitHub repository. 2022.

List of Figures

3.1 High-level system architecture 24
3.2 Jobs table schema 25

List of Tables

2.1 Hardware requirements for the Emotion Detection application 10
2.2 Supported video formats 11
2.3 Environment configuration variables 14
2.4 Upload error messages 20
2.5 Processing error messages 20
2.6 Results error messages 21

3.1 Technology stack by layer 25
3.2 REST API endpoint reference 29
3.3 Test coverage summary 50

List of Codes

2.1 Verify Docker installation 11
2.2 Clone the repository 11
2.3 Start the application with Docker Compose 12
2.4 Stop Docker containers 12
2.5 Navigate to backend 12
2.6 Create virtual environment 12
2.7 Install Python dependencies 13
2.8 Copy environment file 13
2.9 Start Redis (macOS with Homebrew) 13
2.10 Start Celery worker 13
2.11 Start FastAPI server 13
2.12 Navigate to frontend 13
2.13 Install npm dependencies 14
2.14 Start frontend development server 14
3.1 Job model definition (app/models/job.py) 26
3.2 Database configuration (app/core/database.py) 27
3.3 FastAPI application setup (main.py) 28
3.4 Video upload handler (app/api/videos.py) 29
3.5 Celery configuration (app/core/celery_app.py) 30
3.6 Video processing task (app/services/tasks.py) 31
3.7 Face detection implementation 33
3.8 Emotion recognition implementation 34
3.9 Temporal smoothing implementation 35
3.10 Main video processing method 36
3.11 Job polling hook (hooks/useJobPolling.ts) 39
3.12 Analysis results hook (hooks/useAnalysisResults.ts) 39

3.13 API client configuration (services/api.ts) 40
3.14 Core type definitions (types/index.ts) 40
3.15 Emotion timeline visualization (components/EmotionTimeline.tsx) . 42
3.16 Adaptive layout for video orientation 43
3.17 Backend error handling 44
3.18 Task error handling 44
3.19 Frontend error display 45
3.20 Configuration with Pydantic Settings 46
3.21 Example API endpoint test 47
3.22 Pipeline unit test example 47
3.23 React component test example 48
3.24 Running tests 49